

Arquitetura do *protocolo MCP*

Por dentro do MCP: quem conversa com quem (**host**, **cliente** e **servidor**), as duas camadas do protocolo (**dados** e **transporte**), as mensagens **JSON-RPC 2.0** que descobrem e chamam ferramentas e as notificações que mantêm tudo atualizado. Com exemplos reais da especificação atual.

Host · Cliente · Servidor

JSON-RPC 2.0

stdio · Streamable HTTP

server/discover

tools/list · tools/call

Versão 2026-07-28

Formação Claude Code 2026 · Baseado em modelcontextprotocol.io (Architecture overview, Versioning; especificação 2026-07-28) e cursor.com/docs/mcp, consultados em outubro de 2026

Host, cliente *e servidor*

HOST MCP · EX.: CURSOR

Cliente MCP 1

Cliente MCP 2

Cliente MCP 3

Cliente MCP 4

↔ conexão dedicada ↔

Servidor A · local (ex.: sistema de arquivos)

↔ conexão dedicada ↔

Servidor B · local (ex.: banco de dados)

↔ conexão dedicada ↔

Servidor C · remoto (ex.: Sentry) · atende dois clientes

↔ conexão dedicada ↔

HOST

A aplicação de IA que coordena e gerencia um ou vários clientes MCP.

CLIENTE

Componente que mantém a conexão com **um** servidor e busca o contexto que o host vai usar.

SERVIDOR

Programa que oferece contexto aos clientes, rodando local ou remotamente.

Um cliente *por servidor*

O host cria **um cliente MCP para cada servidor**, e cada cliente mantém uma conexão dedicada. O exemplo da documentação oficial usa o VS Code; no Cursor é igual:

- 1 O Cursor (host) conecta ao servidor do **GitHub** e cria o cliente 1.
- 2 Conecta ao servidor do **Playwright** e cria o cliente 2.
- 3 O agente vê as ferramentas dos dois juntas, mas cada chamada vai pelo cliente certo.

"LOCAL" E "REMOTO"

Servidor MCP é o programa que serve o contexto, onde quer que rode. Um servidor com `stdio` roda na mesma máquina e normalmente atende **um** cliente: é o "servidor local". Um servidor com `Streamable HTTP` roda numa plataforma e atende **muitos** clientes: é o "servidor remoto".

Se um servidor cai, os outros continuam funcionando: o Cursor isola as falhas por servidor e marca só aquela chamada como falha.

Duas camadas: *dados e transporte*

CAMADA DE TRANSPORTE (EXTERNA)

Canais de comunicação e autenticação: abre a conexão, enquadra as mensagens e protege a troca entre os participantes.

CAMADA DE DADOS (INTERNA)

O protocolo em **JSON-RPC 2.0**: descoberta de versão e capacidades, as primitivas (tools, resources, prompts) e as notificações.

CAMADA DE DADOS INCLUI	PARA QUÊ
Descoberta	Versões, capacidades e identidade do servidor (<code>server/discover</code>)
Recursos do servidor	Tools, resources e prompts
Recursos do cliente	Elicitation: o servidor pede algo ao usuário
Utilitários	Notificações em tempo real e progresso de operações longas

O transporte esconde os detalhes da conexão: a **mesma mensagem JSON-RPC** vale em qualquer transporte.

stdio e Streamable HTTP

STDIO

Usa a **entrada e a saída padrão** de um processo para falar com processos locais na mesma máquina. Desempenho máximo, sem custo de rede. O Cursor inicia o servidor com o comando do `mcp.json` e conversa pelo stdin/stdout.

Por isso um servidor stdio deve escrever **logs no stderr**, nunca no stdout: o stdout é do protocolo.

STREAMABLE HTTP

HTTP POST para as mensagens do cliente ao servidor, com **Server-Sent Events** opcionais para streaming. Permite servidores remotos e autenticação HTTP padrão: bearer token, chave de API, cabeçalhos. O MCP recomenda **OAuth** para obter os tokens.

NO CURSOR	EXECUÇÃO	IMPLANTAÇÃO	USUÁRIOS	ENTRADA	AUTENTICAÇÃO
<code>stdio</code>	Local	O Cursor gerencia	Um	Comando de shell	Manual
<code>SSE</code>	Local ou remota	Como servidor	Vários	URL do endpoint SSE	OAuth
<code>Streamable HTTP</code>	Local ou remota	Como servidor	Vários	URL do endpoint HTTP	OAuth

O que o servidor *oferece*

PRIMITIVA	O QUE É	MÉTODOS	EXEMPLO
Tools	Funções executáveis para a IA agir	<code>tools/list</code> · <code>tools/call</code>	Consultar o banco, criar uma issue
Resources	Fontes de dados que dão contexto	<code>resources/list</code> · <code>resources/read</code>	Esquema do banco, conteúdo de um arquivo
Prompts	Modelos reutilizáveis de interação	<code>prompts/list</code> · <code>prompts/get</code>	Exemplos de uso das ferramentas

O padrão é sempre o mesmo: **listar** o que existe (`*/list`) e depois **pegar** ou **executar**. Como a lista é consultada na hora, ela pode mudar com o tempo.

O MCP também tem **extensões** opcionais, como **Tasks** (operações longas, consultadas depois) e **Apps** (interface interativa no chat, suportada pelo Cursor).

E o que o cliente *oferece*

ELICITATION

O servidor pede uma **informação a mais ao usuário** ou uma **confirmação** de ação, com o método `elicitation/create` . O Cursor suporta: a pergunta aparece para você no chat.

O cliente declara em cada requisição o que sabe fazer (por exemplo, `"elicitation": {}`). Assim o servidor nunca pede algo que o cliente não consegue atender.

DESCONTINUADO EM 2026-07-28

O QUE FAZER

Sampling (o servidor pede uma resposta ao modelo do cliente)

Ainda na especificação, mas com remoção prevista

Logging (o servidor manda logs ao cliente)

Logar no `stderr` (stdio) ou usar OpenTelemetry

Recursos descontinuados ficam na especificação por pelo menos **doze meses** (ou noventa dias, numa exceção prevista) antes de poderem ser removidos.

Sem estado: *cada requisição se explica*

Na versão atual, o MCP é um protocolo **sem estado**: toda requisição leva no campo `_meta` a **versão do protocolo**, as **capacidades** do cliente e, normalmente, a **identidade** do cliente. O servidor processa cada uma sozinha, sem depender de conexão anterior.

- ✓ Todo servidor implementa `server/discover`: devolve versões aceitas, capacidades e identidade
- ✓ Chamar o `server/discover` é **opcional** para o cliente; é o jeito prático de saber tudo numa requisição só
- ✓ Se a versão não for aceita, o servidor responde com `UnsupportedProtocolVersionError` e a lista de versões; o cliente tenta de novo

RESPOSTA DO SERVER/DISCOVER (RESUMIDA)

```
{
  "jsonrpc": "2.0", "id": 1,
  "result": {
    "supportedVersions": ["2026-07-28"],
    "capabilities": {
      "tools": { "listChanged": true },
      "resources": {}
    },
  },
  "_meta": {
    "io.modelcontextprotocol/serverInfo":
      { "name": "example-server", "version": "1.0.0" }
  }
}
```


Descobrimo *as ferramentas*

REQUISIÇÃO · TOOLS/LIST

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/list",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientInfo":
        { "name": "example-client", "version": "1.0.0" },
      "io.modelcontextprotocol/clientCapabilities":
        { "elicitation": {} }
    }
  }
}
```

RESPOSTA (UMA DAS FERRAMENTAS)

```
{
  "jsonrpc": "2.0", "id": 2,
  "result": { "tools": [ {
    "name": "weather_current",
    "title": "Weather Information",
    "description": "Get current weather information
                  for any location worldwide",
    "inputSchema": {
      "type": "object",
      "properties": {
        "location": { "type": "string" },
        "units": { "type": "string" } },
      "required": ["location"] }
    } ] }
}
```

O **nome**, a **descrição** e o **inputSchema** (JSON Schema dos argumentos) são o que o modelo lê para decidir quando e como usar a ferramenta. É isso que ocupa a categoria "MCP" do contexto.

Chamando *uma ferramenta*

REQUISIÇÃO · TOOLS/CALL

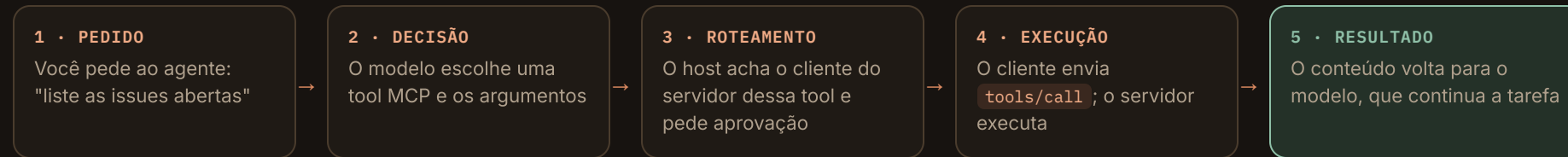
```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "weather_current",
    "arguments": {
      "location": "San Francisco",
      "units": "imperial"
    },
  },
  "_meta": { ... versão, cliente, capacidades ... }
}
```

RESPOSTA

```
{
  "jsonrpc": "2.0", "id": 3,
  "result": {
    "resultType": "complete",
    "content": [ {
      "type": "text",
      "text": "Current weather in San Francisco:
              68°F, partly cloudy..."
    } ]
  }
}
```

O `content` é uma lista com `type` : texto, imagem (em base64) e outros tipos. É assim que um servidor de navegador devolve um screenshot.

Do pedido *à resposta*



PSEUDOCÓDIGO DA DOCUMENTAÇÃO OFICIAL

```
# Pseudo-código: como a aplicação de IA executa uma chamada de ferramenta
async def handle_tool_call(conversation, tool_name, arguments):
    client = app.find_mcp_client_for_tool(tool_name)
    result = await client.call_tool(tool_name, arguments)
    conversation.add_tool_result(result.content)
```

No Cursor, você vê os passos 3 a 5 no chat: o pedido de aprovação, os **argumentos** (seta ao lado do nome da ferramenta) e a **resposta**, que dá para expandir.

Notificações *em tempo real*

- 1 O servidor anuncia em `capabilities` que avisa mudanças:
`"tools": {"listChanged": true}`.
- 2 O cliente abre um canal com `subscriptions/listen`, pedindo
`"toolsListChanged": true`.
- 3 O servidor confirma com
`notifications/subscriptions/acknowledged`.
- 4 Quando as ferramentas mudam, chega
`notifications/tools/list_changed`, e o cliente pede o
`tools/list` de novo.

NOTIFICAÇÃO

```
{
  "jsonrpc": "2.0",
  "method": "notifications/tools/list_changed",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/subscriptionId": 4
    }
  }
}
```

- ✓ **Sem `id`**: notificação não tem resposta (regra do JSON-RPC 2.0)
- ✓ **Opcional**: só chega para quem pediu
- ✓ **Melhor esforço**: pode se perder numa reconexão; o cliente também deve consultar de tempos em tempos

Como o protocolo *evolui*

As versões do MCP são datas no formato `AAAA-MM-DD` : a data da última mudança **incompatível**. Mudanças compatíveis não trocam a versão.

ESTADO	SIGNIFICADO
Draft	Em andamento, ainda não pronta para uso
Current	A versão atual, pronta para uso; pode receber mudanças compatíveis
Final	Versões passadas e completas, que não mudam mais

VERSÃO ATUAL

A versão **current** em outubro de 2026 é a **2026-07-28**, a dos exemplos desta aula. Ela trouxe o modelo sem estado com `server/discover` e descontinuou `sampling` e `logging`.

O ECOSISTEMA

Além da **especificação**, o projeto MCP mantém os **SDKs** por linguagem, o **MCP Inspector** (ferramenta para testar servidores) e os **servidores de referência** (aula 07).

Teste o que *ficou*

1. O Cursor conecta a três servidores MCP. Quantos clientes MCP ele cria?

A) Um

B) Três, um por servidor

C) Nenhum

D) Seis

B. Cada cliente tem uma conexão dedicada.

2. Qual é o formato das mensagens da camada de dados?

A) XML

B) JSON-RPC 2.0

C) GraphQL

D) gRPC

B. O mesmo em qualquer transporte.

3. Que método executa uma ferramenta?

A) tools/list

B) tools/call

C) server/discover

D) prompts/get

B. Com `name` e `arguments`.

4. Por que um servidor stdio não deve escrever logs no stdout?

A) É mais lento

B) O stdout é o canal das mensagens do protocolo

C) O Cursor apaga o arquivo

D) Não há motivo

B. Logs vão para o stderr.

Mensagens simples, *conexões padronizadas*

O **host** (Cursor) cria um **cliente** para cada **servidor**. A camada de **dados** fala JSON-RPC 2.0 sem estado, com `_meta` em toda requisição; a de **transporte** usa stdio ou Streamable HTTP. Ferramentas são descobertas com `tools/list`, executadas com `tools/call` e atualizadas por notificações.

Host · cliente · servidor

Dados · transporte

server/discover

tools/list · tools/call

list_changed

Próxima aula → 07 · Onde Encontrar MCP Disponíveis

Fontes: modelcontextprotocol.io/docs/learn/architecture; modelcontextprotocol.io/specification/versioning (versão 2026-07-28); cursor.com/docs/mcp, consultados em outubro de 2026. Os JSON são os exemplos da documentação oficial, alguns resumidos.